MULTI-VENDOR E-COMMERCE ORDER FULFILLMENT & INVOICE GENERATOR

Object Oriented Programming Using Java & Kotlin (4CTGC2011)

CMR University — School of Engineering & Technology | Project 25 | Academic Year 2026–2027

Beginner-friendly Java console simulation and technical report

Abstract

This project models a marketplace checkout workflow where an order may contain physical products, digital software licenses, and perishable groceries from different vendors. The application applies product-specific tax and fulfillment rules, reserves inventory, and prints an itemized invoice. It demonstrates Java interfaces, abstract classes, inheritance, constructor chaining, polymorphism, encapsulation, custom checked exceptions, threads, and synchronization.

1. Introduction

Online marketplaces combine products with different fulfillment requirements. Physical goods need shipping and tracking, digital products are delivered electronically, and perishable goods may need special packaging. A single invoice workflow should handle these differences while maintaining a common product model. This project implements a console simulation rather than a production e-commerce platform.

2. Problem Statement

Design a Java application that stores mixed product types in a single order, calculates line subtotals and product-specific taxes, computes shipping where applicable, reserves stock safely, tracks order status, and prints a unified invoice. Invalid quantities and unavailable stock must be handled through custom exceptions.

3. Objectives

1) Apply OOP principles. 2) Define the Shippable interface. 3) Create an abstract MarketplaceProduct class. 4) Implement PhysicalProduct, DigitalLicense, and PerishableFoodItem. 5) Use super() constructor chaining. 6) Generate a detailed invoice. 7) Demonstrate thread-based fulfillment and synchronized stock reservation. 8) Handle domain errors with custom checked exceptions.

4. Scope and Assumptions

The project runs locally in a terminal and uses in-memory inventory. It does not process real payments, send real email, persist data, or connect to a courier. GST rates are configurable educational inputs and are not legal or tax advice. Shipping rates are simplified academic rules. Real deployments must validate applicable tax rules, shipping contracts, privacy and security requirements.

5. Functional Requirements

The system must create product objects with validated attributes; support mixed product types in one order; calculate subtotal and tax; calculate shipping only for shippable items; generate tracking IDs for physical/perishable shipments; reserve inventory; update order status; print a unified invoice; reject invalid quantities and insufficient stock; and demonstrate fulfillment in a worker thread.

6. Non-Functional Requirements

Encapsulation: fields are private. Maintainability: product types have separate classes. Correctness: calculations use decimal currency formatting for display. Thread safety: inventory mutations are synchronized. Readability: methods and class names describe their purpose. Limitation: Java double is used for instructional simplicity; production billing should use BigDecimal.

7. System Architecture

The application is organized into a product model, inventory/order workflow, invoice service, and entry point. MarketplaceProduct defines common state and tax behavior. Concrete subclasses override tax calculations and fulfillment behavior. Order stores a list of MarketplaceProduct references. OrderFulfillmentTask uses Inventory from a background thread. InvoiceGenerator calculates totals by invoking polymorphic methods.

8. Class Design

Shippable defines calculateShippingCost() and generateTrackingId(). MarketplaceProduct stores SKU, title, vendor, vendor price and quantity as private final fields, and declares calculateFinalTax() abstract. PhysicalProduct implements Shippable and calculates shipping from weight. DigitalLicense uses zero shipping. PerishableFoodItem can charge chilled packaging. Inventory protects a stock map with synchronized methods. Order holds the customer, items, ID and status.

9. OOP Concepts Demonstrated

Abstraction: MarketplaceProduct hides the general product tax rule. Interface: Shippable defines a contract for delivery-capable products. Inheritance: concrete product classes extend MarketplaceProduct. Polymorphism: InvoiceGenerator invokes calculateFinalTax() through a base-class reference. Encapsulation: product fields are private. Constructor chaining: subclasses call super(...) to initialize common fields. Overriding: each product provides its own tax and shipping behavior.

10. Order Fulfillment and Concurrency

OrderFulfillmentTask implements Runnable. Main creates a Thread, starts it, and calls join() before printing the invoice. Inventory.reserveStock() is synchronized, so each stock check-and-decrement is performed under the same monitor lock. This prevents two threads from simultaneously taking the same last units for a single SKU. A production-quality multi-item order would also need an atomic reservation transaction or rollback if a later line fails.

11. Exception Handling

InvalidQuantityException represents zero or negative quantities. OutOfStockException represents insufficient stock. Both are checked exceptions and are caught by the fulfillment task. Input constructors also validate invalid strings, negative prices and invalid rates using IllegalArgumentException. In a production system, validation exceptions would be mapped to clear UI messages and logged without exposing sensitive data.

12. Invoice Calculation

For each line: line subtotal = unit price × quantity; tax = line subtotal × configured GST rate / 100; shipping is computed according to product fulfillment rules. Grand total = sum of subtotals + sum of taxes + sum of shipping. The invoice prints order ID, customer, date, status, product, quantity, subtotal, tax, shipping, vendor, SKU and tracking/fulfillment details.

13. Example Scenario

The demonstration order contains a Wireless Mouse at Rs. 1,200 with 18% tax and 0.5 kg shipping weight; a Windows 11 Key at Rs. 4,500 with 18% tax and zero shipping; and two units of Fresh Apples at Rs. 200 each with a demonstration 5% tax and chilled packaging. The example rates are illustrative. The generated invoice calculates the line values and displays tracking IDs for shippable products.

14. Testing Plan

Test a valid mixed cart; confirm digital shipping is zero; verify physical shipping formula; check that taxes are calculated by subclass; test zero/negative quantities; request more stock than available; run concurrent reservation attempts against limited stock; verify rejected order status; verify invoice totals equal subtotal + tax + shipping. For stronger evaluation, add automated unit tests and test the multi-line rollback case.

15. Sample Execution

Compile with: javac -d out src/*.java. Run with: java -cp out Main. The program prints a fulfillment confirmation, invoice header, itemized rows, shipping tracking IDs, totals and remaining stock. A second deliberately oversized order demonstrates the OutOfStockException and rejected order status. UUID-based order and tracking IDs vary between runs.

16. Limitations and Future Enhancements

Current limitations include in-memory inventory, simplified GST/shipping assumptions, no payment integration, no database, no actual delivery/email integration, and no transaction rollback for partially reserved multi-item orders. Future versions can add BigDecimal money arithmetic, database transactions, atomic multi-SKU reservation, vendor-wise shipments, PDF invoice export, customer address validation, payment status, cancellation/refund handling, unit tests, and a graphical interface.

17. Conclusion

The project demonstrates how object-oriented design can represent diverse marketplace products while sharing a single order and invoice workflow. It maps to Modules 1–3 through Java classes and constructors, abstraction and polymorphism, custom exceptions, threads and synchronization. The implementation is intentionally small enough for a beginner to explain during a live demonstration.

References

1. CMR University, School of Engineering & Technology, OOP Using Java & Kotlin (4CTGC2011), Academic Project Compendium, Project 25, page 24. 2. Java Platform documentation for classes, interfaces, exceptions and threads (consult the JDK version used in the lab).

Appendix A — Compile and Run

```
javac -d out src/*.java
java -cp out Main
```

Appendix B — Project Files

Java sources, UML PlantUML and text diagrams, README, editable DOCX report and viva Q&A are included in the package.